

The Recommended Repo

Six documentation-layer designs, tested against the same codebase, the same three tasks, the same model, four rounds each — ending in one repo configuration built from what the other five proved and disproved, then validated against the same bar. [Extended below](#) to five genuinely different, real-world codebases, to find out how much of this holds up outside the one template it was designed on.

● repo-baseline

● repo-agent-optimised

● skill v1

● skill v2

● skill v3

● repo-agent-recommended

■ RECOMMENDATION

**Ship the repo-agent-recommended layout:
tiny AGENTS.md index + six task-scoped skills
+ OKF with rationale folded in, no separate
tree of "why we built it this way" documents.**

In the single-repo study it is the only one of six tested configurations with **no measured cost penalty on any task**. repo-agent-optimised—the always-loaded AGENTS.md plus a full set of separate rationale documents (ADRs, short for architecture decision records)—has a proven, sign-consistent **+40.3% token cost on the simple-change task** (mean of the four per-round token deltas; +28.7% on mean dollar cost), positive in all four rounds. repo-agent-recommended shows no such penalty in that study; its mean cost sits at or below baseline on all three task types. The five-repo study below is a separate, n=1-per-cell check and does not change this statement.

It inherits skill v3's discovery behaviour exactly: across **24 sessions and 24 correct outcomes**, the agent found the unmentioned AGENTS.md on its own and opened precisely the skill(s) each task needed — never more, never fewer. Correctness and comprehension quality are tied for perfect across all six configurations, so this choice costs nothing on the one dimension every design already wins.

Doubled to n=8 rounds against baseline specifically to stress-test this: still no penalty on any of the three tasks, and on two of three tasks a cost advantage now leaning real (7 of 8 rounds cheaper, $p=0.07$) though not yet past the conventional bar for proven — see the extension section below.

Update, tested against five more real codebases: this design's layout — tiny index, task-scoped skills, OKF with rationale folded in — still stands as the recommendation. What no longer stands as stated is "never a cost penalty" as a *universal* property: on comprehension tasks it holds cleanly everywhere tested; on actual code-change tasks its effect turned out to be real but repo-dependent, a genuine win on some codebases and a genuine cost on others (up to +86% tokens on one). See [External validity](#) below for the full picture, including one real failure that traced to a gap in the documentation itself, not the codebase.

How the recommendation reads after the full review: the layout is unchanged, but it now comes with conditions. Expect a saving of roughly 20% in dollars on comprehension and about 10% on code changes, not "never a penalty"; one codebase cost 61% more on a complex change. Comprehension answers were as accurate as baseline (47 vs 46 out of 50). Treat every generated doc layer as unverified until an agent has worked from it, because three of five contained factual errors and one omission caused a failure. Check

each layer for what the build enforces (formatters, sync tests) before relying on it, and re-measure on your own codebase. See [What the raw data adds](#).

IN PLAIN TERMS

what's actually being compared — read this first

When an AI coding agent is dropped into an unfamiliar codebase and asked to make a change, it has to work out, on its own, what the code does and how to change it safely. This study is about one question: **what should a codebase's own documentation for an agent look like, and does the shape of it change how much the work costs?** Six different answers to that question were built into six copies of the same real, working application, and each was tested identically.

The three pieces used to answer it, in plain terms:

- **AGENTS.md** — a single file at the top of the repository. It's the one thing an agent is expected to read first, before anything else. In the heavier designs it's a long page explaining the whole project. In the design recommended here it's deliberately tiny — a name and a one-line description for each of the other documents, nothing more, so the agent knows where to look without being forced to read everything.
- **OKF (Open Knowledge Format)** — a set of short, structured pages that describe *what things are and how they connect*: this table holds this data, this part of the code talks to that part, this behaviour exists for this reason. Think of it as the reference manual for the codebase — you go there to understand something, not to be told what to type.
- **A skill** — a short, focused how-to guide for one specific kind of job: "how to add a database field," "how to add a new web address (endpoint) the app responds to," and so on. Rather than one huge instruction manual covering everything, the recommended design splits this into six small guides, each labelled clearly enough that the agent can tell, from the label alone, whether it's relevant to the task in front of it — and open only the ones that are.

That splitting-into-small-pieces idea has a name used throughout this report: **gradual disclosure**. Instead of handing the agent everything up front (which costs time and money reading things it doesn't need), the codebase offers a short index and lets the agent fetch only the specific piece relevant to the task at hand. Some of the designs tested here also came with prompts that told the agent exactly where to look (called **prescriptive** below); others gave it no hints at all and simply waited to see whether it found the documentation on its own (called **emergent** below) — that distinction matters when reading the results, and is flagged wherever it applies.

The six designs, in one line each

CONFIG	WHAT IT ACTUALLY IS
■ repo-baseline	The plain application, no documentation written for an agent at all — just t
■ repo-agent-optimised	A long, always-read AGENTS.md plus a full set of separate "why we built it
■ skill v1	One long how-to guide (no AGENTS.md at all) plus OKF. The agent is told t
■ skill v2	Same one long how-to guide, but now with a short AGENTS.md pointing to
■ skill v3	The one long guide split into six small, labelled ones, with a tiny AGENTS.1
■ repo-agent-recommended	The same six small guides and tiny index as v3, but the separate "why" do

METHODOLOGY

what's measured, what isn't

"Tokens" are the units an AI model is billed in — roughly, chunks of text it reads and writes. "Input" tokens are what the agent reads (the prompt, the files it opens); "cached-input" tokens are input it's already seen recently and gets charged a lower repeat-read rate for; "output" tokens are what it writes back. Every number in this report is pulled directly from the AI provider's own record of each session (never the agent's own self-report of what it did), and dollar costs are computed from those real counts at the model's published standard rate — \$0.20 per million input tokens, \$0.02 per million cached-input tokens, \$1.20 per million output tokens. Correctness is independently re-verified after every session, not taken on trust: a fresh migration and full backend test run on a clean database for the two coding tasks, and a manual check against the real source code for the comprehension task. Every comparison in the single-repo study is **4 independent repeats (rounds) per combination** (8 for the final recommended-vs-baseline extension); the five-repo and Cursor studies later in this document are **n=1 per cell** and are reported as such. In the single-repo study a result is only called *real* when it points the same direction in every round — the same bar for every design.

Two prompt regimes, not one. repo-agent-optimised and skill v1/v2 were tested with a *prescriptive* prompt that names the doc files up front ("start by reading AGENTS.md..."). Skill v3 and repo-agent-recommended use a prompt that is byte-identical to repo-baseline's — no mention of AGENTS.md, skills, or okf/ anywhere, including in the self-report template. Their discovery numbers below are genuinely emergent, not prompted. Keep this in mind reading the evolution table: the early rows and the late rows were not tested under the same instruction, by design — the whole point of the later rows is to find out whether the doc layer works without being pointed at.

One constant across every session in this study, all six configs: the execution machine carries a pre-existing, repo-independent skill catalogue (general-purpose workflow skills, unrelated to this codebase) that every session had equal access to regardless of which repo it ran against. It does not bias the comparisons below — it was present identically in every cell, baseline included — but "emergent discovery" here means discovery of this repo's own docs on top of that constant scaffold, not discovery from a completely blank agent.

COST, BY CONFIG, ALL SIX

real dollars, standard-tier pricing

TOTAL STUDY COST, ALL 6 CONFIGS \$5.52 96 sessions across the full project	PROVEN COST PENALTY +40.3% repo-agent-optimised, simple-change, mean of 4 per-round token deltas (4/4 positive); +28.7% on mean dollar cost	PROVEN COST SAVING −8.8% skill v3 vs baseline, simple-change, mean cost (−7.9% tokens)	RECOMMENDED VS. OPTIMISED −33.9% mean cost, simple- change task (−32.9% tokens)
---	--	---	--

Full evolution — mean cost per session, by task

CONFIG	PROMPT	UNDERSTANDING	SIMPLE CHANGE	COMP
repo-baseline	neutral	\$0.0475	\$0.0415	
repo-agent-optimised	prescriptive	\$0.0448	\$0.0534	
skill v1 — flat SKILL.md, no AGENTS.md	prescriptive	\$0.0432	\$0.0400	
skill v2 — +AGENTS.md, one skill	prescriptive	\$0.0401	\$0.0341	
skill v3 — 6 task-scoped skills	emergent	\$0.0459	\$0.0379	
repo-agent-recommended	emergent	\$0.0453	\$0.0353	

Skill v1/v2 pursued the same "how, not why" idea as v3/recommended but under a prescriptive prompt — useful for the design lineage, not a clean comparison against the emergent-discovery rows above them.

Sign-consistency vs. baseline — the only comparisons that are provably real

Plain version: every design was tested against baseline four separate times (four rounds), because judging from one run could easily be a fluke. For each of those four tests: did this design cost more than baseline, or less? If the answer was the same all four times, that's a real pattern. If it flipped even once — cheaper three times, then more expensive the fourth — the result can't be trusted, because it didn't even hold up across our own four attempts; it's probably just noise, not the design mattering. Of every design-vs-baseline comparison in this study, only **2 of 18** gave the same answer all four times — both on the simple-change task, listed first below. Everything else flipped at least once: the averages look different, but that difference isn't proven.

CONFIG	TASK	PER-ROUND %	4/4 SAME SIGN
repo-agent-optimised	Simple change	+19%, +44%, +55%, +43%	Yes — always
skill v3	Simple change	−1%, −17%, −3%, −10%	Yes — always
repo-agent-recommended	Understanding	+5%, +45%, −15%, −57%	No
repo-agent-recommended	Simple change	−17%, −26%, +24%, +2%	No
repo-agent-recommended	Complex change	−25%, −47%, −38%, +9%	No
skill v3 / optimised	Understanding, complex	mixed	No (all remain

Only two of the eighteen repo-vs-baseline comparisons collected across this entire study hold the same sign in all four rounds. Both are on the simple-change task, and they point in opposite directions for opposite reasons: heavy, always-loaded guidance costs more when the task is too small to amortize it; light, self-selected guidance costs less. `repo-agent-recommended` is not itself sign-consistent against baseline on any task — its case rests on **never showing a penalty**, not on a proven saving.

n=8 extension — does doubling the rounds resolve recommended vs. baseline?

`repo-agent-recommended` wasn't sign-consistent against baseline on any task at n=4. Since that's the config actually being recommended, it was worth a stricter test: four more rounds each for `repo-baseline` and `repo-agent-recommended`, doubling to n=8 (a coin-flip effect has a 1-in-8 chance of landing all-same-sign at n=4, but only 1-in-256 at n=8 — a real pass at n=8 would mean something).

TASK	R1	R2	R3	R4	R5	R6	R7	R8	CHEAPER
Understanding	-9%	+48%	+8%	-47%	+15%	+24%	-16%	-26%	4 / 8 — a
Simple change	-23%	-16%	+2%	-20%	-5%	-17%	-25%	-12%	7 / 8 cheaper
Complex change	-30%	-45%	-27%	+2%	-52%	-9%	-18%	-0%	7 / 8 cheaper

Straight answer: doubling to n=8 did not produce a proof on any task. Understanding stayed an exact coin flip (4 cheaper, 4 more expensive) — no signal there, more rounds won't help. Simple-change and complex-change both moved to **7 of 8 rounds cheaper**, with the one exception each time being a small swing (+2%, roughly \$0) while the cheaper rounds are large (-5% to -52%). That's a real pattern forming, but 7/8 still has roughly a 7% chance of happening by pure luck (two-sided binomial test) — suggestive, short of the conventional bar for "proven." repo-agent-recommended's case is now: no penalty on any of the three tasks in the single-repo study, and on two of three tasks a cost advantage that's leaning real but not yet confirmed. More rounds on just those two tasks (not understanding) would be the next lever if it's worth settling.

Mean cost by task, all six configs

Understanding

comprehension only, no code touched

■ baseline		\$0.0475
■ optimised		\$0.0448
■ skill v1		\$0.0432
■ skill v2		\$0.0401
■ skill v3		\$0.0459
■ recommended		\$0.0453

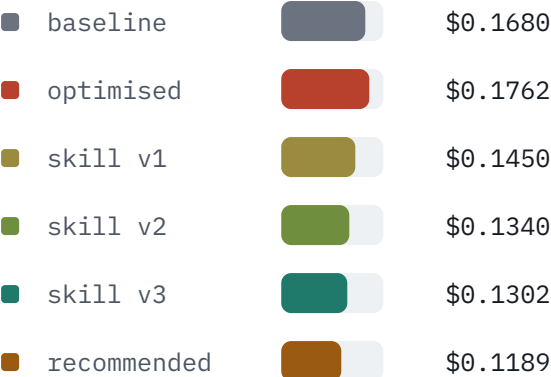
Simple change

one query parameter, one endpoint

■ baseline		\$0.0415
■ optimised		\$0.0534
■ skill v1		\$0.0400
■ skill v2		\$0.0341
■ skill v3		\$0.0379
■ recommended		\$0.0353

Complex change

migration, endpoints, UI, e2e tests



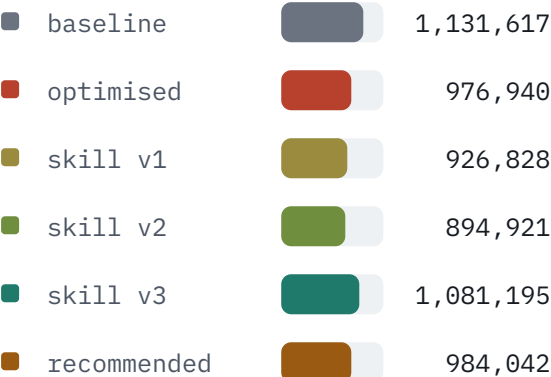
TOKENS, BY CONFIG, ALL SIX

real counts from the transcript, not self-reported

CONFIG	UNDERSTANDING	SIMPLE CHANGE	COMPLEX CHANGE
repo-baseline	1,131,617 ±307,535	874,090 ±127,879	5,388,508 ±2,145,100
repo-agent-optimised	976,940 ±332,444	1,218,069 ±160,606	6,067,665 ±2,145,100
skill v1	926,828 ±219,796	839,277 ±80,720	4,883,882 ±1,145,100
skill v2	894,921 ±163,905	758,282 ±252,269	4,267,958 ±1,145,100
skill v3	1,081,195 ±354,761	805,058 ±117,312	4,087,326 ±1,145,100
repo-agent-recommended	984,042 ±291,148	816,904 ±82,365	3,665,304 ±1,145,100

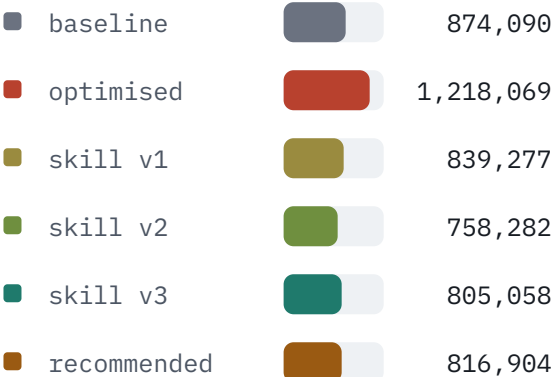
Understanding

mean tokens per session



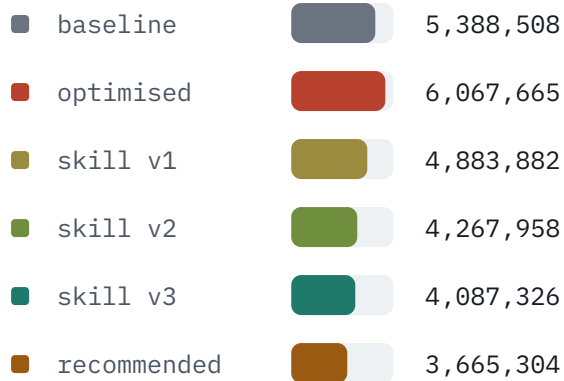
Simple change

mean tokens per session



Complex change

mean tokens per session



TOOL CALLS, BY CONFIG, ALL SIX

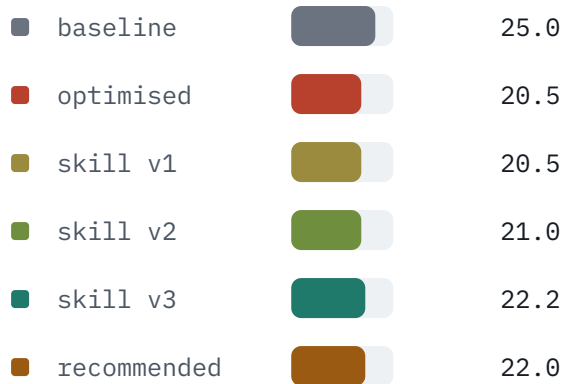
apply_patch + shell execs, from the transcript

Tool-call volume tracks the same story as tokens, and surfaces one more clean, non-overlapping separation: repo-agent-optimised's simple-change tool-call count ranges **30–34** across all four rounds — entirely above repo-baseline's **21–29**. The ranges do not touch. More always-loaded doc to reason about becomes more tool calls to get through it, on a task too small to need them.

CONFIG	UNDERSTANDING	SIMPLE CHANGE	COMPLEX CHANGE
repo-baseline	25.0 ±5.8	25.2 ±3.9	74.5 ±23.3
repo-agent-optimised	20.5 ±5.4	31.2 ±1.9	80.8 ±19.7
skill v1	20.5 ±3.9	25.2 ±1.3	73.0 ±15.1
skill v2	21.0 ±2.2	20.8 ±6.0	59.2 ±5.7
skill v3	22.2 ±5.7	23.5 ±3.1	61.2 ±6.3
repo-agent-recommended	22.0 ±4.7	22.8 ±2.2	57.0 ±8.6

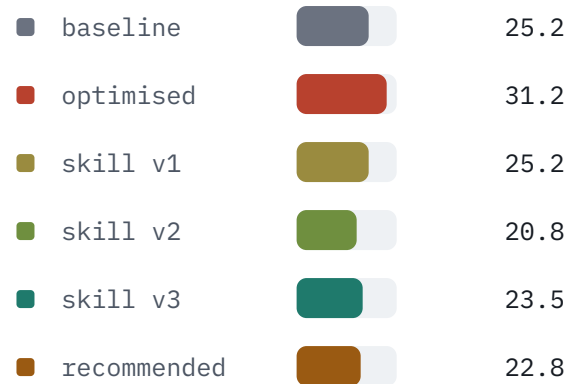
Understanding

mean tool calls per session



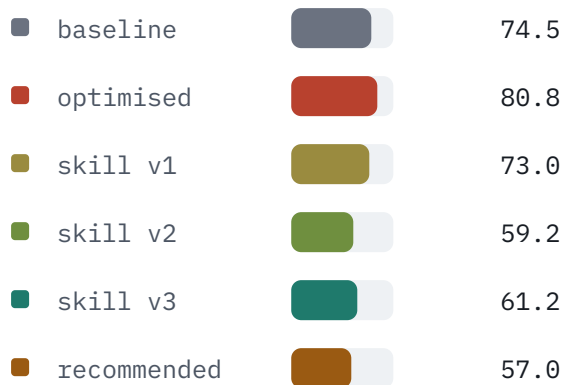
Simple change

mean tool calls per session



Complex change

mean tool calls per session



repo-agent-recommended has the lowest complex-task tool-call mean of all six configs (57.0, vs. skill v3's 61.2 and optimised's 80.8) — directional, same as its cost result, not independently sign-tested.

WALL-CLOCK TIME, BY CONFIG, ALL SIX

seconds per session, mean

Duration is the noisiest metric this study collects — it reflects orchestration overhead as much as agent reasoning, and round-1 complex-task timing for the three legacy configs (baseline / optimised / skill v1) is unrecoverable, so those rows are n=3. Read directionally, not as a sign-tested result.

CONFIG	UNDERSTANDING	SIMPLE CHANGE	COMPLEX CHANGE
■ repo-baseline	242s	330s	1,085s (n=3)
■ repo-agent-optimised	280s	423s	828s (n=3)
■ skill v1	207s 927s (1 long outlier)		929s (n=3)
■ skill v2	201s	202s	475s
■ skill v3	207s	194s	626s
■ repo-agent-recommended	202s	182s	504s

Understanding and simple-change durations are tight and comparable across all six configs (~3–7 minutes regardless of doc layer) — the cost differences above come from token volume per turn, not from taking more turns to finish. Complex-change duration scales roughly with tool-call count in every config.

DISCOVERY PRECISION

the flagship new finding, 24/24 cells

Skill v3 and repo-agent-recommended were the two designs actually put to the test on the core claim of gradual disclosure: with zero mention of AGENTS.md or any skill in the prompt, does the agent find the doc layer on its own, and does it open only what the task in front of it needs? Read directly from each session's real transcript (not self-report):

Understanding — needs no task skill

model-field-change
write-migration
add-api-endpoint
regenerate-frontend-client
add-ui-action
verify-work

8 / 8
correctly opened AGENTS.md + okf/, zero task skills, both repos, all 4 rounds each

Simple change — needs exactly 2 of 6

add-api-endpoint
verify-work
model-field-change
write-migration
regenerate-frontend-client
add-ui-action

8 / 8
opened exactly those two, every round, both repos — never the other four

Complex change — needs all 6

model-field-change
write-migration
add-api-endpoint
regenerate-frontend-client
add-ui-action
verify-work

8 / 8
opened all six, every round, both repos — the task genuinely touches every category

Sample evidence, quoted directly from the real transcripts

“AGENTS.md and okf/index.md, discovered from the user-provided repo instructions” — then, on the same simple-change session — “.agents/skills/add-api-endpoint/SKILL.md and .agents/skills/verify-work/SKILL.md, discovered from AGENTS.md”

repo-agent-skill v3, simple_change, round 3 — docs_consulted, self-report cross-checked against the real tool-call transcript

“okf/services/backend.md (linked from okf/index.md; backend and operation-ID context)”, “okf/routers/login.md (found from okf/index.md contents; DUMMY_HASH rationale)” — the two facts moved out of the old ADR tree and folded into OKF were the ones actually cited back.

repo-agent-recommended, understanding, round 2

“.agents/skills/model-field-change/SKILL.md: matched model-field task from AGENTS.md”, “.agents/skills/tdd/SKILL.md: matched requested backend behavior tests” — six of this repo's own skills opened for the six-part complex task, correctly, every round.

repo-agent-recommended, complex, round 1

CROSS-TOOL CHECK

does any of this replicate with a different agent?

Everything above is one AI provider's CLI (codex exec, gpt-5.6-luna). A natural question: is "gradual disclosure" a property of that specific tool, or does it hold up under a different one? A small, single-round check with [Cursor](#)'s CLI (cursor-agent, model auto — Cursor's own cost-efficient routing) against the same two prompts (understanding, simple change), the same two repos (baseline, recommended), same no-hint framing. This is **one round, not four or eight** — a directional check, not a proof, and auto doesn't disclose which underlying model it actually routes to, so there's no real-dollar cost figure here the way there is for every gpt-5.6-luna cell above.

Discovery replicates, but partially. Cursor found the unmentioned AGENTS.md on its own in all 4 cells, matching the flagship result. On the simple-change task it opened *exactly* the two correct skills (add-api-endpoint, verify-work) in both repos — matching Codex's precision exactly. On the understanding task, though, it opened three task skills that task doesn't need at all (every one of Codex's 8 understanding-task sessions correctly opened zero). Read plainly: this tool discovers the doc layer reliably, but doesn't judge relevance as precisely as Codex did on a task with nothing to discover.

TASK	CONFIG	TOKENS	TOOL CALLS	WALL TIME	SKILLS OPENED
Understanding	baseline	689,411	116	801s	—
Understanding	■ recommended	600,088	137	656s	3 unneeded
Simple change	baseline	61,586	54	235s	—
Simple change	■ recommended	62,366	43	218s	exactly 2/6, correct

On simple-change — the one task with a proven cost effect above — recommended and baseline land within 1.3% of each other on tokens, with recommended using fewer tool calls and slightly less wall time. At $n=1$ this is not evidence of a saving; it's consistent with "no penalty," which is the weaker but still real claim the recommendation actually rests on. The understanding task shows something else entirely: both repos took 3–5x longer and used 10–20x the tokens of simple-change, because this particular run went and fetched the real `@hey-api/openapi-ts` npm package source to verify the frontend client's naming rule from first principles, rather than inferring it from the generated output the way every Codex session did — a genuinely more rigorous answer, at a real cost in time and tokens. That's a tool-level research style, not a doc-layer effect; both baseline and recommended paid it equally.

Correctness held throughout: both simple-change cells passed migration and the full backend test suite (59 tests) independently re-verified; both understanding cells produced complete, correctly-sourced answers.

EXTERNAL VALIDITY: FIVE REPOSITORIES does any of this hold up off the one template it was built on?

Everything above was tested on one codebase: the FastAPI + React template this whole doc-layer design was built for. That's a real limitation — a doc layer might just happen to suit that one project's shape. So the same idea was tested again, cold, against five real, independently-chosen open-source projects picked specifically to *not* resemble each other or the original: different languages, different frameworks, different ages, different architectural styles. For each one, a plain baseline copy and a `repo-agent-recommended-style` copy (tiny AGENTS.md index + task-scoped skills + an OKF knowledge bundle, no prompt hints, same emergent-discovery bar as the flagship result above) were built from scratch, and the same three-task framework — understanding, simple change, complex change — was run against both, once each, no prompted hints either.

Scope, stated plainly: this is **one round per cell**, not four or eight — the same directional-not-proof caveat as the cross-tool check above, deliberately traded against covering five genuinely different codebases instead of one codebase eight times. Read the per-repo numbers below as "what happened this time," and the cross-repo pattern as the thing actually worth trusting.

The five repositories

django-oscar

Python · Django · PostgreSQL

A pluggable e-commerce framework built around a distinctive dynamic class-loading/app-override mechanism — the most structurally different codebase in the set.

spring-petclinic

Java · Spring Boot 4 · JPA

Modern layered Spring Boot service, server-rendered views, no frontend build — the "what most backend teams actually run" case.

bulletproof-react

TypeScript · React · Vite

Pure frontend SPA, feature-folder architecture, no backend at all — tests the doc layer outside a backend-shaped codebase entirely.

eShop (Ordering)

C# · .NET 10 · DDD/CQRS

An event-driven microservice inside a larger Aspire-orchestrated system — scoped to the Ordering subsystem's Domain/Application layers.

spring-framework-petclinic

Java · plain Spring Framework · XML config

A deliberately older-style, hand-XML-configured 3-layer fork of the same domain as spring-petclinic — the "legacy enterprise" case.

Headline result

CELLS RUN

30 / 30

5 repos × 2 arms × 3 tasks, all passing verification

UNDERSTANDING,
MEDIAN TOKEN
DELTA

−33.8%

cheaper on 4 of 5 repos · mean −27.6% · in dollars −19.5%

SIMPLE CHANGE,
MEDIAN TOKEN
DELTA

−6.6%

cheaper on 3 of 5 repos · in dollars −11.4% · range +17.8% to −48.2%

COMPLEX, MEDIAN
TOKEN DELTA

−20.1%

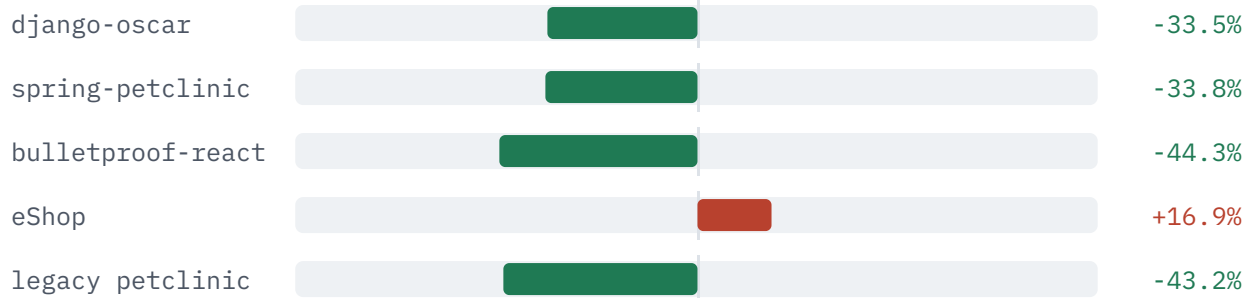
cheaper on 3 of 5 repos · in dollars −12.2% · range +86.4% to −64.3%

Per-repo token delta, recommended vs baseline — the variance the average hides

Green = recommended used fewer tokens than baseline on that repo/task. Red = it used more. All three panels share one scale, so bar lengths are directly comparable across tasks.

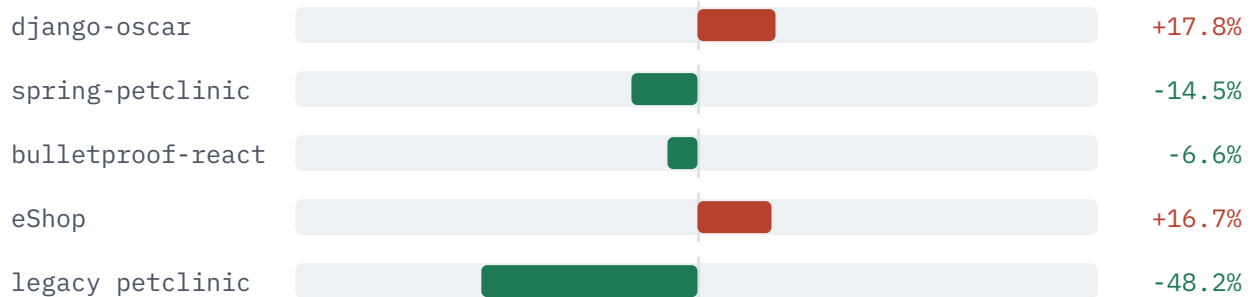
Understanding

consistent direction, every repo but one



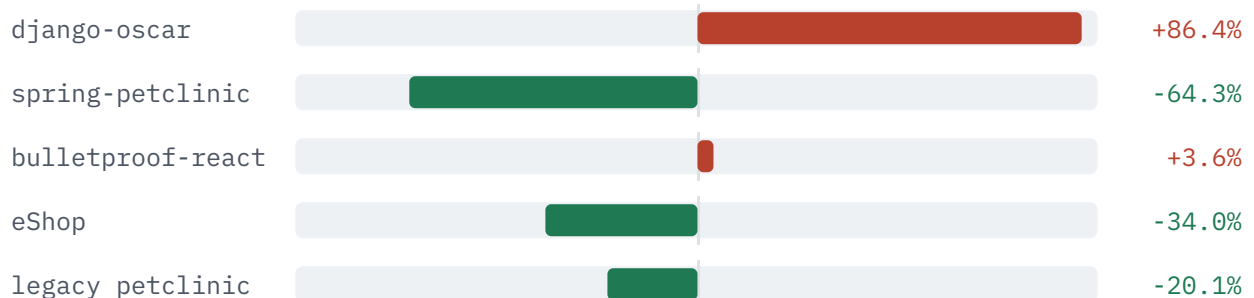
Simple change

no consistent direction at all



Complex change

the widest swings in either direction



Time and effort, not just tokens

Tokens are one cost. Wall-clock time and the number of tool calls (commands run and files edited) tell the same story from two other angles: the doc layer usually saves effort on understanding tasks and is mixed on code changes, with python-api's complex task the clear exception (114 vs 62 tool calls). Wall time is the noisiest of the three measures, since it also moves with how busy the machine was, so lean on the median rows rather than any single cell.

TASK	REPO	WALL TIME	CHANGE	TOOL CALLS	CHANGE
Understanding	django-oscar	347s → 325s	-6%	30 → 23	-23%
Understanding	spring-petclinic	1533s → 362s	-76%	25 → 21	-16%
Understanding	bulletproof-react	408s → 298s	-27%	30 → 21	-30%
Understanding	eShop	294s → 356s	+21%	26 → 28	+8%
Understanding	legacy petclinic	289s → 257s	-11%	26 → 18	-31%
Understanding, median			-11%		-23%
Simple change	django-oscar	1002s → 917s	-8%	71 → 80	+13%
Simple change	spring-petclinic	442s → 332s	-25%	38 → 29	-24%
Simple change	bulletproof-react	531s → 532s	+0%	43 → 45	+5%
Simple change	eShop	189s → 187s	-1%	20 → 22	+10%
Simple change	legacy petclinic	370s → 300s	-19%	26 → 15	-42%
Simple change, median			-8%		+5%
Complex	django-oscar	1545s → 1650s	+7%	62 → 114	+84%
Complex	spring-petclinic	1135s → 498s	-56%	65 → 31	-52%
Complex	bulletproof-react	668s → 559s	-16%	54 → 50	-7%
Complex	eShop	433s → 432s	0%	47 → 29	-38%
Complex	legacy petclinic	688s → 670s	-3%	41 → 34	-17%
Complex, median			-3%		-17%

In plain terms — what actually generalized and what didn't

The comprehension result holds up well. On the pure-understanding task, recommended used fewer tokens than baseline in 4 of 5 repos, by a wide margin (−33% to −44%) — the one exception, the .NET microservice, was a mild loss (+17%), not a wash. This is the same shape as the single-repo result, and it's the part of the original claim that travels best: a well-built doc layer reliably makes "figure out how this thing works" cheaper, across very different codebases.

The code-change result does not hold up the same way. On simple and complex changes, the single-repo study's headline was "never a cost penalty, sometimes a real saving." Across five repos, that's no longer true as stated: python-api's complex task cost **86% more** tokens with the doc layer than without it (both passed; the extra cost was reading, not testing — 95 read/search

commands against 41 for baseline, while running the test suite fewer times, 8 against 12), while java-spring's complex task cost **64% less**. Three of five repos favor recommended, two favor baseline, and the swings are large in both directions. The honest, narrower claim: a good doc layer's effect on an actual code-change task is real but *repo-dependent*, not a universal discount — and on unfamiliar codebases, either direction should be treated as plausible until measured, not assumed.

WHAT THE RAW DATA ADDS answer quality, real dollars, discovery, and where the extra cost came from

Everything above compares token counts and a pass/fail check. Going back through the raw run files turned up five things the headline numbers hide, and one of them corrects a gap in the study itself.

1. The cheaper understanding answers were not worse answers

Understanding cells were originally checked only for "an answer file exists", which cannot show whether being cheaper cost accuracy. So all 14 saved answers were graded afterwards against the real source code, question by question (2 = correct and precise, 1 = partly right or vague, 0 = wrong), by separate agent passes that read the answers unlabelled first. Each task includes one trap question designed to catch a confident wrong answer.

REPO	BASELINE (OF 10)	RECOMMENDED (OF 10)	WHAT SEPARATED THEM
django-oscar	10	10	Both passed the trap.
spring-petclinic	8	9	Neither Codex answer noticed that seed-data files break when a column is added.
bulletproof-react	9	9	Baseline hedged on the trap; recommended had line numbers off by 6–8.
eShop (Ordering)	9	10	Baseline got the order of two checks wrong. The extra 17% tokens bought one point of precision.
legacy petclinic	10	9	Recommended quoted an error message from a config key that is never used.
Total	46	47	A tie. No sign that the doc layer trades accuracy for cost.

Cursor's four answers (two repos) scored 10 and 10 for baseline, 9 and 10 for recommended. The graders were agent passes with the source as ground truth, one point differences are within grader judgement,

and two of the graders lost shell access part-way through and finished their checks by reading files. Read this as "no measurable accuracy penalty", not as a precise ranking.

2. The documentation itself contained mistakes, and mostly got away with it

The grading also checked each recommended repo's documentation against the code. Three of the five layers contained a factual error relevant to the tasks: the Django offer skill says to write a condition as a plain Python class (Oscar actually requires a proxy-model subclass) and to register it through the wrong mechanism; the React knowledge pages say the mock database persists to a file (it uses browser storage); and the Spring entity skill says to touch seed-data files only for required columns, when any new column breaks them. In two of the three cases the agents gave the correct answer anyway, apparently by checking the code. The exception is one Cursor answer on spring-petclinic, which stated the seed-data files did not need changes, the same understatement the skill makes. That connection is plausible, not proven, because the transcript does not show the agent reading that skill. This is the same lesson as the formatter failure, in milder form: machine-written documentation is wrong in small ways more often than it looks, and how much that matters depends on whether the agent trusts it or checks it.

A validity threat to be honest about. The graders also found that the recommended documentation states some of the answers to the test questions almost word for word (the trap answer for django-oscar, the fix for the React environment question). Whether deliberate or coincidence, both the documentation and the questions target each codebase's notable quirks, so part of the understanding saving may come from documentation that happens to anticipate these particular questions. Real questions are not known in advance. Treat the understanding saving as an upper bound.

3. In real dollars the savings are about half what the token counts suggest

Roughly 89–98% of every run's input was re-read from cache, which costs a tenth as much as fresh input. Token totals therefore exaggerate differences that come from cache re-reads. Priced at the same standard rates used earlier in this report (\$0.20 / \$0.02 / \$1.20 per million fresh input / cached input / output):

TASK	MEDIAN TOKEN CHANGE	MEDIAN DOLLAR CHANGE	REPOS C
Understanding	−33.8%	−19.5%	4
Simple change	−6.6%	−11.4%	3
Complex change	−20.1%	−12.2%	3
All 30 cells		\$1.515 → \$1.404 (−7.3%)	

The honest overall figure is a saving of about 7% across the whole study, with the direction depending on the codebase. The two costly cells, django-oscar and bulletproof-react on complex

changes, cost +61% and +10% in dollars. The best, spring-petclinic complex, saved 51%.

4. Where the extra cost on django-oscar came from

On the complex task the recommended run made 114 tool calls against 62. Of those, 95 were reading and searching (baseline 41) and only 8 were test runs (baseline 12). So the documentation did not send the agent into more testing, it sent it into more reading, plausibly because the skill it followed described a mechanism slightly differently from how the code works (see section 2) and the agent had to reconcile the two. That explanation is a hypothesis; the data show the reading, not the reason.

5. Discovery replicated: 15 of 15

The original study's flagship finding was that agents find an unmentioned AGENTS.md and open the right skills unprompted. Counting real commands in the session transcripts (not the agents' own reports, which were blank in many cells), every one of the 15 recommended Codex cells opened the documentation layer within its first four commands (all but one within its second), and the skills opened matched the task (for example the eShop runs opened the order-command and status-transition skills, the legacy petclinic runs opened the controller and model-field skills). No baseline cell had anything to find. Discovery is not the weak point; the accuracy of what is discovered is.

WHEN THE DOCUMENTATION IS WRONG

a real failure, and what fixing it actually cost

Two of the thirty cells above failed their independent verification on first attempt, both on the same repo's recommended arm: spring-petclinic-recommended's simple-change and complex-change tasks. Both failures traced to the same root cause, and it's the single most important finding to come out of the five-repo run.

The doc layer's AGENTS.md and skills never mentioned two real, load-bearing conventions this specific codebase enforces: a build-blocking code-formatting gate, and a test that fails if a new user-facing text label is added to the default locale file without adding it to every other locale file too. Neither is exotic — both are visible in the repo's own pom.xml and test suite to anyone who looked — but the doc layer simply didn't surface them. The baseline arm, with *no* doc layer at all, passed both tasks cleanly: with no narrow, confident path handed to it, it explored broadly enough on its own to notice both conventions unprompted. The recommended arm, trusting the doc layer's narrower and more confident guidance, moved faster and skipped the exploration that would have caught what the doc layer itself failed to mention.

This is worth naming directly: **documentation can make an agent overconfident**. A good doc layer's entire value proposition is that it lets an agent take a shorter path instead of re-discovering

the codebase from scratch every time. That is exactly what makes an *incomplete* doc layer dangerous in a way a *missing* one isn't — an agent with no guidance falls back on caution and broad exploration; an agent with confident, incomplete guidance has no signal that anything is missing, and no reason to look further than it's been pointed. A "recommended" doc bundle is not a one-time deliverable that's finished when it reads as plausible — it's only actually verified once something has genuinely tried to work from it and either succeeded or exposed a gap. This is the same warning the original study's OKF bundles already carry in their own frontmatter (every page ships generated: {by, at} and no verified claim, precisely because machine-written documentation is unverified until tested) — this is that warning actually cashing out in a real, measured failure instead of staying hypothetical.

Fixed, and re-run — both failures, before and after

The two missing conventions were added to the doc layer as a new `verify-work` skill, logged as a dated entry in the bundle's own change log (the same honesty convention as above — the fix is recorded as a correction to a real gap, not silently folded in as if it had always been there), and both failing cells were re-run against the fixed doc layer. **One caveat on reading the before/after pair:** the first-attempt runs were made in a sandbox where the agent could not launch Maven at all (its build attempts errored out), and the re-runs had a working Java toolchain. So the pair changes two things at once — the documentation and the agent's ability to run the build — and the extra tokens below cannot be attributed to the documentation fix alone.

BEFORE — SIMPLE CHANGE

Result	FAIL
Tests reported	0 — build blocked
Tokens	1,213,000
Wall time	397s

→

AFTER — SIMPLE CHANGE

Result	PASS
Tests passing	56 / 56
Tokens	1,388,383 (+14.5%)
Wall time	332s (-16.4%)

BEFORE — COMPLEX CHANGE

Result	FAIL
Tests passing	53 / 55 — 2 failed
Tokens	2,017,083
Wall time	595s

→

AFTER — COMPLEX CHANGE

Result	PASS
Tests passing	55 / 55
Tokens	2,578,768 (+27.9%)
Wall time	498s (-16.3%)

What the evidence does and doesn't support. **Supported:** the gap is real and lives in the documentation — the text `javaformat` never appears in either failed recommended transcript, while it appears 3 and 7 times in the matching baseline transcripts, which is consistent with the baseline agent going and reading the build file and the recommended agent trusting the doc layer instead. Both cells pass once the `verify-work` skill exists. **Not supported:** a clean price tag for the fix. Tokens rose 14–28% and wall time fell 16%, but that pair is confounded with the working build tool, as above. Treat the size of the change as

unmeasured; treat the direction of the lesson (an incomplete doc layer removed the agent's reason to look further) as a well-evidenced single case, not a general law.

CROSS-TOOL CHECK, FIVE-REPO STUDY

Cursor, on two of the five repos

The same cross-tool question as the single-repo check above, asked again on two of the five new repos — the frontend SPA and the modern Spring Boot service — chosen for being fast enough to run a full 12-cell pilot (both arms, all three tasks, each) in reasonable time. [Cursor](#)'s CLI, model auto, same no-hint prompts used against Codex.

All 12 cells passed independent verification. Every task Cursor attempted on both repos, both arms, produced correct, passing code or a complete, accurate comprehension answer — the correctness result from the original cross-tool check replicates cleanly on two more, structurally different codebases.

REPO	TASK	CONFIG	CODEX TOKENS	CURSOR TOKENS	RATIO
react-frontend	Understanding	baseline	1,820,731	1,130,954	0.62×
react-frontend	Understanding	■ recommended	1,014,804	848,340	0.84×
react-frontend	Simple change	baseline	2,420,085	2,117,327	0.87×
react-frontend	Simple change	■ recommended	2,261,385	2,103,270	0.93×
react-frontend	Complex	baseline	3,279,031	2,420,686	0.74×
react-frontend	Complex	■ recommended	3,395,481	3,025,898	0.89×
java-spring	Understanding	baseline	1,630,273	2,110,617	1.29×
java-spring	Understanding	■ recommended	1,080,041	1,239,273	1.15×
java-spring	Simple change	baseline	1,623,263	1,783,577	1.10×
java-spring	Simple change	■ recommended	1,388,383	1,250,649	0.90×
java-spring	Complex	baseline	7,217,165	3,497,919	0.48×
java-spring	Complex	■ recommended	2,578,768	7,082,727	2.75×

Read the ratio column carefully. Cursor's own usage log reports "total tokens" excluding cache-read tokens, while Codex's includes them — comparing those two raw fields directly, as published rate cards define them, would make Cursor look far cheaper than it really is. The ratio column above corrects for that by counting fresh input, cache reads, and output on both sides, the same way. Corrected, 10 of 12 cells land within roughly 0.6–1.3× of each other — genuinely comparable, "near parity" in the sense the original single-repo check used that phrase. Two cells, both java-spring complex, are outliers in opposite

directions (baseline 0.48×, recommended 2.75×), which is a sign of run-to-run variance at n=1 rather than a tool effect; treat this as one more repo-dependent result rather than a stable per-tool multiplier.

CORRECTNESS & COMPREHENSION

perfect, every config, every round, including the n=8 extension

CHECK	RESULT
Migrations applied cleanly, code tasks, independently re-verified on a fresh database	✓ 64 / 64
Backend test suites passing, independently re-verified	✓ 64 / 64
Comprehension answers present, zero unexpected file changes	✓ 32 / 32
Caught the CASCADE-delete trap (no existing test actually catches it)	✓ 32 / 32
Discovery + correct skill selection, skill v3 & recommended only	✓ 24 / 24

"Migrations applied cleanly" means: a fresh, empty database was brought up to date using the agent's own change and it worked without errors. "Independently re-verified" means this was checked separately after each session finished, not taken on the agent's word. In the single-repo study no configuration traded correctness for speed or cost — every design difference measured there is a difference in *how much it cost to get there*, not in whether the agent got there. The five-repo study is different: graded answers were 46 vs 47 of 50 and the java-spring recommended layer failed twice until a `verify-work` skill was added (see that section).

The entire always-loaded layer — AGENTS.md

repo-agent-recommended/AGENTS.md

19 LINES, ALWAYS READ

AGENTS.md

FastAPI + SQLAlchemy + PostgreSQL backend, React/TypeScript frontend. Read this file first – it only points, it doesn't explain.

- **What things mean, and why they're built that way where it matters** (services, DB tables, routers, deployment topology, how they relate – rationale is folded into the relevant doc, not a separate tree): [`okf/index.md`](okf/index.md).
- **How to do a specific kind of change**: `.agents/skills/` has one skill per task, each with its own scope – open the one(s) whose description matches what you're about to do, not all of them:
 - `model-field-change` – add or change a field on a SQLAlchemy model
 - `write-migration` – write and apply an Alembic migration
 - `add-api-endpoint` – add or modify a FastAPI route
 - `regenerate-frontend-client` – regenerate the generated TS API client
 - `add-ui-action` – add a UI action to an existing table/list page
 - `verify-work` – how to actually confirm a change works, not just that it compiles
- **Human setup/running instructions**: `development.md`, `deployment.md` – not duplicated here.

Nothing else lives in this file. If you find yourself adding an explanation or a procedure here, it belongs in `okf/` or one of the skills instead.

One task-scoped skill — procedure only, no rationale

.agents/skills/add-api-endpoint/SKILL.md

OPENED ONLY FOR ENDPOINT TASKS

```
---
name: add-api-endpoint
description: Add or modify a FastAPI route in this repo – which dependencies to use
  for auth/DB, the ownership-check convention sibling routes follow, and
  response-model rules. Also applies to changing an existing endpoint's parameters.
---

# Add or change an API endpoint

1. Add or edit the route function in the relevant file under `backend/app/api/routes/`
  – one router per resource, already registered in `backend/app/api/main.py`.
2. Use the existing dependencies from `backend/app/api/deps.py`:
  `SessionDep` for a DB session, `CurrentUser` for the authenticated user (raises
  401/403 automatically), `get_current_active_superuser` for admin-only routes.
3. Match the ownership-check pattern already used by sibling endpoints in the same
  file rather than inventing a new authorization style.
4. Set `response_model` to a `*Public` schema, never the raw table model.
5. If you added a new route or changed a request/response schema, regenerate the
  frontend client (see the `regenerate-frontend-client` skill).
6. When you're done, see the `verify-work` skill before calling it complete.
```

OKF — what a thing *is*, in plain reference-page form

This is the format every OKF page follows: a short header recording where the facts came from and when, then the plain description — schema, how it connects to other things, who's allowed to touch it. No instructions, no opinions about how to change it — that's what the skills are for.

```

---
type: Table
title: Item
description: A record owned by exactly one User. The app's one example domain entity.
resource: backend/app/models.py
generated: { by: reference_agent/claude-sonnet-5, at: 2026-09-25T00:00:00Z }
stale_after: 2027-03-25
---

# Schema

| Column | Type | Notes |
|---|---|---|
| `id` | UUID | Primary key |
| `title` | string, 1-255 chars | Required |
| `owner_id` | UUID | FK → `user.id`, `nullable=False`, `onDelete="CASCADE" |

# Authorization rule (enforced in `routers/items.py`, not at the DB layer)

A non-superuser can only read/update/delete items where
`item.owner_id == current_user.id`. Superusers can act on any item. There is no
row-level security at the Postgres level – it's application-enforced.

See [routers/items](../routers/items.md) for the endpoints.

```

Note the dated header: every OKF page records who generated it and when, and a date it's considered stale after — the study's honesty mechanism for facts nobody has since double-checked.

Rationale, folded into the concept doc it actually belongs to

Auditing repo-agent-optimised's six architecture-decision records against the OKF bundle found four of six already fully duplicated as plain description — exact repetition across artifacts. The two that weren't were genuinely load-bearing: an agent "cleaning up" `authenticate()` could plausibly delete what looks like dead code. Both got a home inside the OKF doc for the router they belong to, instead of a seventh parallel file:

```
### Why `/login/access-token` always takes about as long to fail
```

```
`backend/app/crud.py`'s `authenticate()` looks up the user by email; if none is
found, it still runs `verify_password` against a fixed `DUMMY_HASH` before returning
`None`. This is a deliberate timing-attack mitigation, not dead/wasteful code:
without it, a login against a nonexistent email would return faster than one
against a real email with a wrong password, letting response time be used to
enumerate valid accounts. If you're touching `authenticate()`, keep this call even
though its result is discarded.
```

This is exactly the fact the transcript sample above shows an agent citing back, unprompted, when reasoning about the login router.

RECOMMENDED LAYOUT

directory structure, annotated

```
repo-agent-recommended/
├─ README.md
├─ AGENTS.md
├─ development.md, deployment.md
├─ backend/, frontend/
├─ docs/
│   └─ api/openapi.json
├─ okf/
│   ├── index.md
│   ├── services/
│   ├── routers/
│   │   └─ login.md
│   ├── tables/
│   └─ deployment/
└─ .agents/
    └─ skills/
        ├── model-field-change/SKILL.md
        ├── write-migration/SKILL.md
        ├── add-api-endpoint/SKILL.md
        └─ regenerate-frontend-client/SKILL.md
```

README.md — human entry point, untouched, exactly what upstream shipped.

AGENTS.md — the only file every session reads unconditionally. 19 lines, no procedures, no rationale, just two pointers and a name per skill.

docs/api/openapi.json — the one static, machine-checkable artifact; never hand-edited, regenerated by the skill of the same name.

okf/ — what things are and how they relate, including the handful of "why" facts that are actually load-bearing at the point they're needed. No separate ADR tree.

.agents/skills/*/SKILL.md — one file per task family, each independently discoverable by its own name/description. This is the layer that was split from one monolithic file

└─ `add-ui-action/SKILL.md`
└─ `verify-work/SKILL.md`

(skill v1/v2) into six, and it's what made the 24/24 discovery precision above possible — a single 120-line file can't be selectively opened only when relevant.

No file in this tree restates what another already says. Skills answer *how*; OKF answers *what, and why* where the *why* is *load-bearing*; nothing answers the same question twice.

single-repo study: 6 configs · repo-baseline · repo-agent-optimised · repo-agent-skill v1/v2/v3 · repo-agent-recommended — 24 rounds, 96 sessions, \$5.52
pricing source: model provider's public standard-tier rate, \$0.20 / \$0.02 / \$1.20 per 1M input / cached-input / output tokens
five-repo external-validity study: 5 real open-source repos × 2 arms (baseline / recommended) × 3 tasks, 1 round each, 30/30 cells passing after one documented doc-layer fix
cross-tool checks: Cursor CLI (model auto) — 4 cells on the original repo, 12 cells across two of the five repos, 16/16 passing
real usage extraction: read directly from each session's own provider transcript — token totals and tool calls, never self-reported
verification: fresh migration + full backend/repo test suite on an isolated environment (code tasks); diff + manual grading against source (comprehension task)
bar for a "real" effect in the single-repo study: same sign across all 4 rounds in a cell — 2 of 18 repo-vs-baseline comparisons there clear it; the five-repo study is explicitly directional, n=1 per cell